DATA STRUCTURE

SORTING:-

Q1. Bubble sort

```
#include <iostream>
using namespace std;

class BubbleSort
{
   int A[10], n;

public:

   BubbleSort()
   {
      n = 0;

      for(int i = 0; i < 10; i++)
      {
         A[i] = 0;
      }
```

```
}

void getdata()
{
   cout << "Enter number of elements: ";
   cin >> n;

   cout << "Enter elements: ";

   for(int i = 0; i < n; i++)
   {
      cin >> A[i];
   }
}

void BSC()
{
   for(int i = 0; i < n - 1; i++)
   {
      for(int j = 0; j < n - 1 - i; j++)
      {
```

```
            if(A[j] > A[j + 1])
            {
               int t = A[j];
               A[j] = A[j + 1];
               A[j + 1] = t;
            }
         }
      }
}

void display()
{
   cout << "Sorted elements: ";

   for(int i = 0; i < n; i++)
   {
      cout << A[i] << " ";
   }
}

void process()
```

```
    {
        BSC();
    }
};

int main()
{
    BubbleSort b;

    b.getdata();
    b.process();
    b.display();

    return 0;
}
```

Output:

main.c... Output

```
Enter number of elements: 5
Enter elements: 220 30 40 50 60
Sorted elements: 30 40 50 60 220
```

Q2. Selection sort

```
#include <iostream>
using namespace std;
```

```
class SelectionSort
{
   int A[10], n;

public:

   SelectionSort()
   {
      n = 0;

      for(int i = 0; i < 10; i++)
      {
         A[i] = 0;
      }
   }

   void getdata()
   {
      cout << "Enter number of elements: ";
      cin >> n;
```

```
    cout << "Enter elements: ";

    for(int i = 0; i < n; i++)
    {
        cin >> A[i];
    }
}

void SSC()
{
    for(int i = 0; i < n - 1; i++)
    {
        int min = i;

        for(int j = i + 1; j < n; j++)
        {
            if(A[j] < A[min])
            {
                min = j;
            }
```

```
        }

        int t = A[i];
        A[i] = A[min];
        A[min] = t;
    }
}

void display()
{
    cout << "Sorted elements: ";

    for(int i = 0; i < n; i++)
    {
        cout << A[i] << " ";
    }
}

void process()
{
    SSC();
```

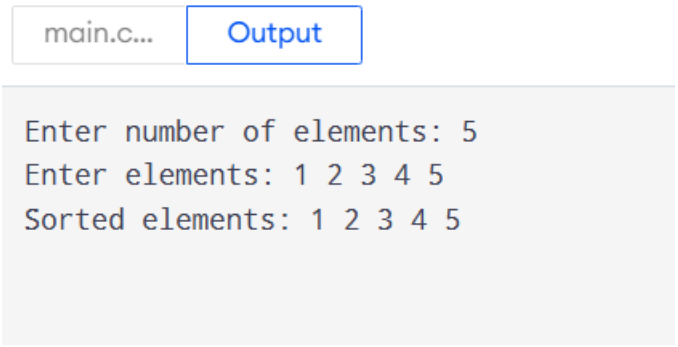
```
    }
};

int main()
{
    SelectionSort s;

    s.getdata();
    s.process();
    s.display();

    return 0;
}
```

Output:



Q3. Insertion sort

```
#include <iostream>

using namespace std;
```

```
class InsertionSort
{
   int A[10], n;

public:

   InsertionSort()
   {
      n = 0;

      for(int i = 0; i < 10; i++)
      {
         A[i] = 0;
      }
   }

   void getdata()
   {
      cout << "Enter number of elements: ";
      cin >> n;
```

```
    cout << "Enter elements: ";

    for(int i = 0; i < n; i++)
    {
        cin >> A[i];
    }
}

void ISC()
{
    for(int i = 1; i < n; i++)
    {
        int key = A[i];
        int j = i - 1;

        while(j >= 0 && A[j] > key)
        {
            A[j + 1] = A[j];
            j--;
        }
```

```
            A[j + 1] = key;
        }
    }

    void display()
    {
        cout << "Sorted elements: ";

        for(int i = 0; i < n; i++)
        {
            cout << A[i] << " ";
        }
    }

    void process()
    {
        ISC();
    }
};

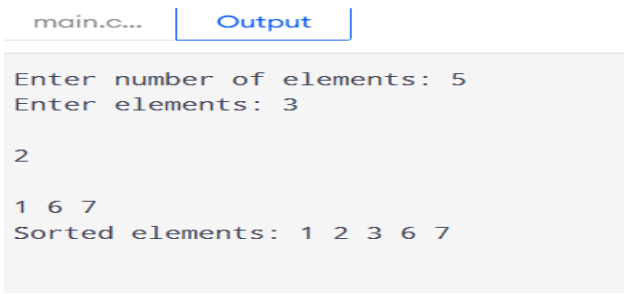
int main()
```

```
{
   InsertionSort s;

   s.getdata();
   s.process();
   s.display();

   return 0;
}
```

Output:



Q4. Quick sort

```
#include <iostream>
using namespace std;

class QuickSort
{
   int A[10], n;

public:
```

```
QuickSort()
{
   n = 0;

   for(int i = 0; i < 10; i++)
   {
      A[i] = 0;
   }
}

void getdata()
{
   cout << "Enter number of elements: ";
   cin >> n;

   cout << "Enter elements: ";

   for(int i = 0; i < n; i++)
   {
      cin >> A[i];
```

```
    }
}

int partition(int ll, int ul)
{
    int start, end, pivot;

    pivot = A[ll];
    start = ll + 1;
    end = ul;

    while(start <= end)
    {
        while(start <= ul && A[start] <= pivot)
        {
            start++;
        }

        while(A[end] > pivot)
        {
            end--;
```

```
        }

        if(start < end)
        {
            int t = A[start];
            A[start] = A[end];
            A[end] = t;
        }
    }

    int t = A[ll];
    A[ll] = A[end];
    A[end] = t;

    return end;
}

void quicksort(int ll, int ul)
{
    int arr_loc;
```

```
   if(ll < ul)
   {
      arr_loc = partition(ll, ul);

      quicksort(ll, arr_loc - 1);
      quicksort(arr_loc + 1, ul);
   }
}

void display()
{
   cout << "Sorted elements: ";

   for(int i = 0; i < n; i++)
   {
      cout << A[i] << " ";
   }
}

void process()
{
```

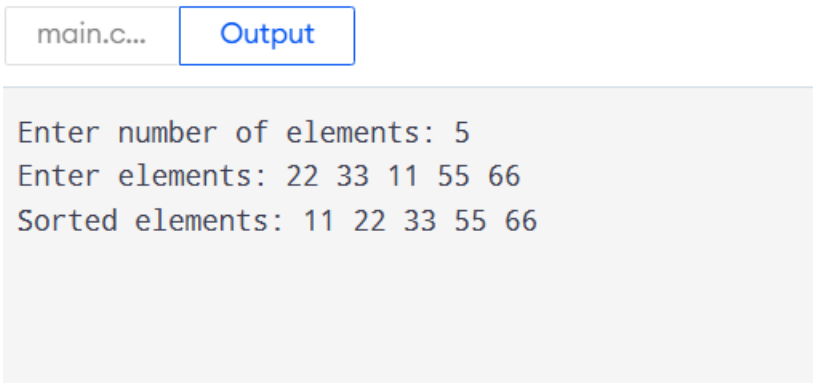
```
        quicksort(0, n - 1);
    }
};

int main()
{
    QuickSort q;

    q.getdata();
    q.process();
    q.display();

    return 0;
}
```

Output:



Q5. Merge sort

```
#include <iostream>
using namespace std;

class MergeSort
{
   int A[10], B[10], n;

public:

   MergeSort()
   {
      n = 0;

      for(int i = 0; i < 10; i++)
      {
         A[i] = 0;
         B[i] = 0;
      }
   }

   void getdata()
```

```
{
   cout << "Enter number of elements: ";
   cin >> n;

   cout << "Enter elements: ";

   for(int i = 0; i < n; i++)
   {
      cin >> A[i];
   }
}

void merge(int low, int mid, int high)
{
   int i = low, j = mid + 1, k = low;

   while(i <= mid && j <= high)
   {
      if(A[i] <= A[j])
      {
         B[k] = A[i];
```

```
        i++;
        k++;
    }
    else
    {
        B[k] = A[j];
        j++;
        k++;
    }
}

while(i <= mid)
{
    B[k] = A[i];
    i++;
    k++;
}

while(j <= high)
{
    B[k] = A[j];
```

```
        j++;
        k++;
    }

    for(i = low; i <= high; i++)
    {
        A[i] = B[i];
    }
}

void sort(int low, int high)
{
    if(low < high)
    {
        int mid = (low + high) / 2;

        sort(low, mid);
        sort(mid + 1, high);

        merge(low, mid, high);
    }
```

```
    }

    void display()
    {
        cout << "Sorted elements: ";

        for(int i = 0; i < n; i++)
        {
            cout << A[i] << " ";
        }
    }

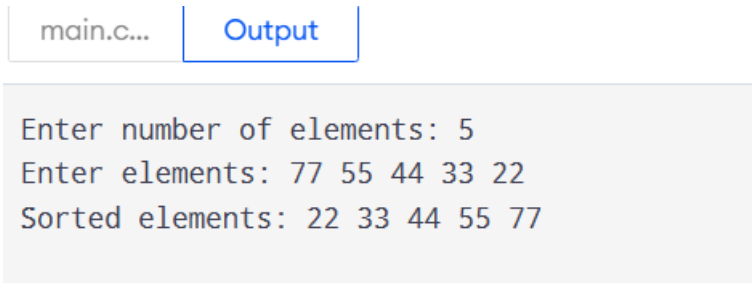
    void process()
    {
        sort(0, n - 1);
    }
};

int main()
{
    MergeSort m;
```

```
    m.getdata();
    m.process();
    m.display();

    return 0;
}
```

Output:



Q. heap sort

```
#include <iostream>
using namespace std;

class HeapSort
{
    int A[10], n;

public:
```

```
HeapSort()
{
   n = 0;

   for(int i = 0; i < 10; i++)
   {
      A[i] = 0;
   }
}

void getdata()
{
   cout << "Enter number of elements: ";
   cin >> n;

   cout << "Enter elements: ";

   for(int i = 0; i < n; i++)
   {
      cin >> A[i];
```

```
    }
}

void heapify(int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if(left < n && A[left] > A[largest])
    {
        largest = left;
    }

    if(right < n && A[right] > A[largest])
    {
        largest = right;
    }

    if(largest != i)
    {
```

```
        int t = A[i];
        A[i] = A[largest];
        A[largest] = t;

        heapify(n, largest);
    }
}

void heapsort()
{
    for(int i = n / 2 - 1; i >= 0; i--)
    {
        heapify(n, i);
    }

    for(int i = n - 1; i > 0; i--)
    {
        int t = A[0];
        A[0] = A[i];
        A[i] = t;
```

```
            heapify(i, 0);
        }
    }

    void display()
    {
        cout << "Sorted elements: ";

        for(int i = 0; i < n; i++)
        {
            cout << A[i] << " ";
        }
    }

    void process()
    {
        heapsort();
    }
};

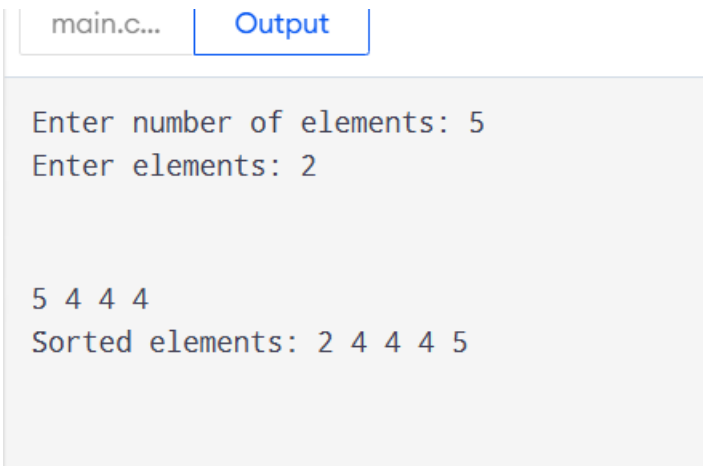
int main()
```

```
{
   HeapSort h;

   h.getdata();
   h.process();
   h.display();

   return 0;
}
```

Output:



SEARCHING:-

Linear search

```
#include <iostream>

using namespace std;
```

```
class LinearSearch
{
   int A[10], n, key;

public:

   LinearSearch()
   {
      n = 0;
      key = 0;

      for(int i = 0; i < 10; i++)
      {
         A[i] = 0;
      }
   }

   void getdata()
   {
      cout << "Enter number of elements: ";
```

```
    cin >> n;

    cout << "Enter elements: ";

    for(int i = 0; i < n; i++)
    {
        cin >> A[i];
    }

    cout << "Enter element to search: ";
    cin >> key;
}

void search()
{
    int flag = 0;

    for(int i = 0; i < n; i++)
    {
        if(A[i] == key)
        {
```

```
            cout << "Element found at position: " << i + 1;
            flag = 1;
            break;
          }
       }

       if(flag == 0)
       {
          cout << "Element not found";
       }
    }
};

int main()
{
    LinearSearch l;

    l.getdata();
    l.search();

    return 0;
```

}

Output:

main.c... Output

```
Enter number of elements: 4
Enter elements: 3 4 2 1
Enter element to search: 4
Element found at position: 2
```

BINARY SEARCH

```
#include <iostream>

using namespace std;


class BinarySearch

{

   int A[10], n, key;


public:


   BinarySearch()

   {

      n = 0;

      key = 0;


      for(int i = 0; i < 10; i++)
```

```
    {
        A[i] = 0;
    }
}

void getdata()
{
    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements in sorted order: ";

    for(int i = 0; i < n; i++)
    {
        cin >> A[i];
    }

    cout << "Enter element to search: ";
    cin >> key;
}
```

```
void search()
{
   int low = 0, high = n - 1, mid;
   int flag = 0;

   while(low <= high)
   {
      mid = (low + high) / 2;

      if(A[mid] == key)
      {
         cout << "Element found at position: " << mid + 1;
         flag = 1;
         break;
      }

      else if(A[mid] < key)
      {
         low = mid + 1;
      }
```

```
        else
        {
            high = mid - 1;
        }
      }

      if(flag == 0)
      {
         cout << "Element not found";
      }
   }
};

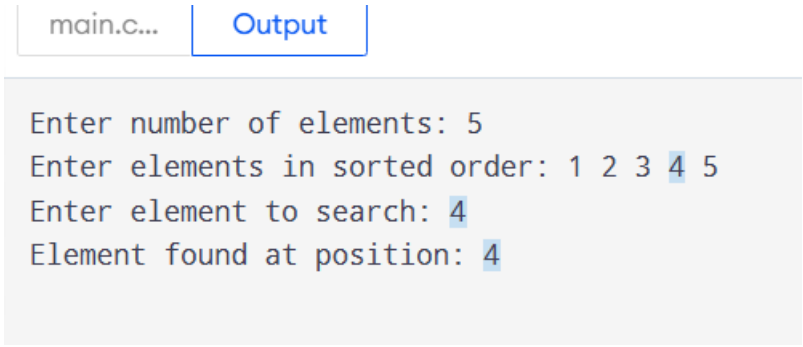
int main()
{
   BinarySearch b;

   b.getdata();
   b.search();

   return 0;
```

```
}
```

Output:


main.c...
Output
Enter number of elements: 5
Enter elements in sorted order: 1 2 3 4 5
Enter element to search: 4
Element found at position: 4